# CSC 452

# FORMAL MODELS OF COMPUTATION

**CMP 452Formal Models of Computation (3 Units) (L: 45)**

Introduction; TOC, Roles of models in computation, Automata theory: Finite state Automata, Push-down Automata, Formal Grammars, Parsing, Relative powers of formal models. Basic computability: Turing-Machines, Universal Turing-Machines, Church's thesis, Solvability and Decidability.

## 1 Introduction

In theoretical computer science, the Theory of Computation (TOC) is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches:

i. Automata theory,
ii. Computability theory and
iii. Computational complexity theory.

In order to perform a rigorous study of computation, computer scientists' work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

### 1.1 Why we need to study the Theory of Computation (TOC)

This lead to theory of what can be computed and what cannot. It involves sketch of some theoretical frameworks that can inform the design of programs to solve a wide variety of problems. Two major reasons are:

i. The shelf life of programming tools
ii. Applications of the theory are everywhere

### *i. The Shelf Life of Programming Tools*

Implementations come and go. In the somewhat early days of computing, programming meant knowing how to write code like. Machine code and Assembly Code, then 1n 1957, Fortran appeared and made it possible for people to write programme that looked more straightforward like mathematics, then later Java changed the way we write program. Along the way, other programming languages became popular, at least within some circles.

Today's programmers can't read code from 50 years ago. Programmers from the early days could never have imagined what a program of today would look like. In the face of that kind of change, what does it mean to learn the science of computing?

The answer is that **there are mathematical properties**, both of problems and of algorithms for solving problems that depend on neither the details of today's technology nor the programming fashion *du jour (of the day in French)*. The theory will address some of those properties. Most of what we will discuss was known by the early 1970s (barely the middle ages of computing history). But it is still **useful in two key ways:**

1. It provides a set of abstract structures that are useful for solving certain classes of problems. These abstract structures can be implemented on whatever hardware/software platform is available.
2. It defines provable limits to what can be computed, regardless of processor speed or memory size. An understanding of these limits helps us to focus our design effort in areas in which it can pay off, rather than on the computing equivalent of the search for a perpetual motion machine.

The **goal** will be to discover fundamental properties of the problems themselves:

1. Is there any computational solution to the problem? If not, is there a restricted but useful variation of the problem for which a solution does exist?
2. If a solution exists, can it be implemented using some fixed amount of memory?

### *ii.* Applications of the Theory Are Everywhere

Computers have revolutionized our world. They have changed the course of our daily lives, the way we do science, the way we entertain ourselves, the way that business is conducted, and the way we protect our security. Some of the application areas of TOC are:

a. It enables both machine/machine and person/machine **communication**. Without them, none of today's applications of computing could exist. E.g. Network communication protocols are languages. Most Web pages are described using the Hypertext Markup Language, HTML. Music can be viewed as a language. And specialized languages enable composers to create new electronic music. Even very unlanguage-like things, such as sets of pictures, can be viewed as languages by, for example, associating each picture with the program that drew it.

b. Both the design and the **implementation of modern programming languages** rely heavily on the theory of context-free languages. Context-free grammars are used to document the languages' syntax and they form the basis for the parsing techniques that all compilers use.

c. People use **natural languages**, such as English, to communicate with each other. Since the advent of word processing, and then the Internet, we now type or speak our words to computers. So we would like to build programs to manage our words, check our grammar, search the World Wide Web, and translate from one language to another. Programs to do that also rely on the theory of context-free languages.

d. Systems as diverse as **parity checkers**, vending machines (e.g. ATM), communication protocols, and building security devices can be straightforwardly described as finite state machines. E.g. A family of network communication protocols are modeled as finite state machines. An example of a simple building security system, modeled as a finite state machine, etc.

e. Many interactive video games are (large, often nondeterministic) finite state machines.

f. DNA is the language of life. DNA molecules, as well as the proteins that they describe, are strings that are made up of symbols drawn from small alphabets (nucleotides and amino acids, respectively). So computational biologists exploit many of the same tools that computational linguists use. For example, they rely on techniques that are based on both finite state machines and context-free grammars.

g. Security is perhaps the most important property of many computer systems. The undecidability results show that there cannot exist a general purpose method for automatically verifying arbitrary security properties of programs. The complexity results serve as the basis for powerful encryption techniques.

h. Artificial intelligence programs solve problems in task domains ranging from medical diagnosis to factory scheduling. Various logical frameworks have been proposed for representing and reasoning with the knowledge that such programs exploit. The undecidability results that show that there cannot exist a general theorem prover that can decide, given an arbitrary statement in first order logic, whether or not that statement follows from the system's axioms.

## 1.2 Models of computation: purpose and types

Almost any plausible statement about computation is **true** in some model, and **false** in others! A rigorous definition of a model of computation is essential when proving negative results: "impossible...". **Special purpose models** vs. **universal models** of computation (can simulate any other model). ***Algorithm*** and ***computability*** are originally intuitive (describes something or knowing something through feelings rather than conscious reasoning or evidence) concepts. They can remain intuitive as long as we only want to show that some specific result can be computed by following a specific algorithm. Almost always an informal explanation suffices to convince someone with the requisite background that a given algorithm computes a specified result. Everything changes if we wish to show that a desired result is ***not computable*.** The question arises immediately: "What tools are we allowed to use?" The attempt to prove negative results about the nonexistence of certain algorithms forces us to agree on a rigorous definition of *algorithm*.

Mathematics has long investigated problems of the type: what kind of objects can be constructed, what kind of results can be computed using a **limited set of primitives and operations**. For example, the question of what polynomial equations (equation formed with variables, non-negative integer exponents and coefficients, set equal to zero) can be solved using radicals, i.e. using addition, subtraction, multiplication, division, and the extraction of roots, has kept mathematicians busy for centuries.

Whenever the tools allowed are restricted to the extent that “intuitively computable” objects cannot be obtained using these tools alone, we speak of a **special-purpose model of computation**. Such models are of great practical interest because the tools allowed are tailored to the specific characteristics of the objects to be computed, and hence are efficient for this purpose. Many examples are close to hardware design. From the theoretical point of view, however, **universal models of computation** are of prime interest. This concept arose from the natural question "What can be computed by an algorithm, and what cannot?". It was studied during the 1930s by Emil Post (1897–1954), Alonzo Church (1903-1995), Alan Turing (1912–1954), and other logicians. They defined various formal models of computation, such as **production systems, recursive functions, the lambda calculus, and Turing machines,** to capture the intuitive concept of "computation by the application of precise rules". All these different formal models of computation turned out to be equivalent.

This fact greatly strengthens **Church's thesis** that the intuitive concept of algorithm is formalized correctly by any one of these mathematical systems. The models of computation defined by these logicians in the 1930s are **universal** in the sense that they can compute anything computable by any other model, given unbounded resources of time and memory. This concept of universal model of computation is a product of the 20-th century which lies at the center of the theory of computation.

The standard universal models of computation were designed to be conceptually simple: Their primitive operations are chosen to be as weak as possible, as long as they retain their property of being universal computing systems in the sense that they can simulate any computation performed on any other machine. It usually comes as a surprise to novices that the set of primitives of a universal computing machine can be so simple, as long as these machines possess two essential ingredients: *unbounded memory* and *unbounded time*.

Two examples of universal models of computation:

a. **Markov algorithms**, which access data and instructions in a sequential manner, and might be the architecture of choice for computers that have only sequential memory.
b. A **simple random access machines** (RAM), based on a random access memory, whose architecture follows the von Neumann design of stored program computers.

Once one has learned to program basic data manipulation operations, such as moving and copying data and pattern matching, the claim that these primitive "computers" are universal becomes believable. Most simulations of a powerful computer on a simple one share three characteristics:

a. It is straightforward in principle,
b. it involves laborious coding in practice, and
c. it explodes the space and time requirements of a computation.

The weakness of the primitives, desirable from a theoretical point of view, has the consequence that as simple an operation as integer addition becomes an exercise in programming. The purpose of these examples is to support the idea that conceptually simple models of computation are equally powerful, in theory, as models that are much more complex, such as a high-level programming language. Unbounded **time** and **memory** is the key that lets the snail ultimately cover the same ground as the hare.

The theory of computability was developed in the 1930s, and greatly expanded in the 1950s and 1960s. Its basic ideas have become part of the foundation that any computer scientist is expected to know. But computability theory is not directly useful. It is based on the concept "computable in principle" but offers no concept of "feasible in practice". And feasibility, rather than "possible in principle", is the touchstone of computer science. Since the 1960s, a theory of the complexity of computation has being developed, with the goal of partitioning the range of computability into complexity classes according to time and space requirements. This theory is still in full development and breaking new ground, in particular with (as yet) exotic models of computation such as quantum computing or DNA computing.

## 1.3 Roles of models in computation

We should recall that models of computation define the theoretical limits of what computers can solve, serving as foundational blueprints for algorithm design, language syntax and hardware architecture. It helps a lot in those things listed above. In addition, they provide essential abstraction for mapping problems to solutions, allowing researchers to simulate complex systems and predict behaviours using tools like Turing machines, s=finite – state machines etc.

Other roles can be categorised as follows:

i. **Defining Computability and limits:** This determine what problems can be solved and at what cost (Time/Memory), distinguishing between decidable and un-decidable problems.
ii. **System Simulation and prediction:** They act as proxies for real-world systems, enabling scientists to conduct simulations, such as modelling disease spread or analysing physical phenomena.
iii. **Programming Language and Compiler Design:** Models establish syntax and semantics, directly influencing how programming languages as structured and interpreted.
iv. **Hardware and Algorithm Design:** They guide the development of physical components (e.g. VLSI chips) and parallel architectures (e.g. PRAM).
v. **Optimization:** Computational models help analyse the effect of parameter values on outcomes to find optimal design solutions.

## 1.4 Types of Computational Models:

**Finite State Machines (FSMs):** Ideal for simple control logic and pattern matching, but limited by finite memory.

**Turing Machines:** The standard theoretical model for defining what is algorithmically computable.

**Random- Access Machine (RAM):** A model closely approaximating modern computer architecture.

**Parallel Random-Access Machine (PRAM):** Used to study parallel algorithms.

# 2 Introduction to formal proof:

## 2.1 Basic Symbols used:

U – Union
∩ - Conjunction
□ - Empty String
Φ – NULL set
**7**- negation
**‘** – compliment
= > implies

**Additive inverse:** a+(-a)=0

**Multiplicative inverse:** a*1/a=1

Universal set U={1,2,3,4,5}

Subset A={1,3}

A’ ={2,4,5}

**Absorption law:** AU(A ∩ B) = A, A ∩ (AUB) = A

**De Morgan’s Law:**

(AUB)’ =A’ ∩ B’

(A ∩ B)’ = A’ U B’

Double compliment

(A’)’ =A

A ∩ A’ = Φ

**Relations:**

Let a and b be two sets a relation R contains aXb.

Relations used in TOC:

**Reflexive:** a = a

**Symmetric:** aRb = > bRa

**Transition:** aRb, bRc = > aRc

If a given relation is reflexive, symmentric and transitive then the relation is called equivalence relation.

## 2.2 Deductive proof:

Deductive proof consists of sequence of statements whose truth lead us from some initial statement called the hypothesis or the give statement to a conclusion statement.

The theorem that is proved when we go from a hypothesis $H$ to a conclusion $C$ is the statement "if $H$ then $C$." We say that $C$ is *deduced* from $H$.

## 2.3 Direct proof (AKA) Constructive proof:

If $p$ is true then $q$ is true

E.g.: if a and b are odd numbers then product is also an odd number.

Odd number can be represented as 2n+1

a=2x+1, b=2y+1

product of a X b = (2x+1) X (2y+1)

= 2(2xy+x+y)+1 = 2z+1 (odd number)

## 2.4 Proof by Contrapositive:

easier to prove. The *contrapositive* of the statement "if $H$ then $C$" is "if not $C$ then not $H$." A statement and its contrapositive are either both true or both false, so we can prove either to prove the other.

**Theorem 1.10:** $R \cup (S \cap T) = (R \cup S) \cap (R \cup T)$.

| | Statement | Justification |
|---|---|---|
| 1. | $x$ is in $R \cup (S \cap T)$ | Given |
| 2. | $x$ is in $R$ or $x$ is in $S \cap T$ | (1) and definition of union |
| 3. | $x$ is in $R$ or $x$ is in both $S$ and $T$ | (2) and definition of intersection |
| 4. | $x$ is in $R \cup S$ | (3) and definition of union |
| 5. | $x$ is in $R \cup T$ | (3) and definition of union |
| 6. | $x$ is in $(R \cup S) \cap (R \cup T)$ | (4), (5), and definition of intersection |

Figure 1.5: Steps in the "if" part of Theorem 1.10

| | Statement | Justification |
|---|---|---|
| 1. | $x$ is in $(R \cup S) \cap (R \cup T)$ | Given |
| 2. | $x$ is in $R \cup S$ | (1) and definition of intersection |
| 3. | $x$ is in $R \cup T$ | (1) and definition of intersection |
| 4. | $x$ is in $R$ or $x$ is in both $S$ and $T$ | (2), (3), and reasoning about unions |
| 5. | $x$ is in $R$ or $x$ is in $S \cap T$ | (4) and definition of intersection |
| 6. | $x$ is in $R \cup (S \cap T)$ | (5) and definition of union |

Figure 1.6: Steps in the "only-if" part of Theorem 1.10

To see why "if $H$ then $C$" and "if not $C$ then not $H$" are logically equivalent, first observe that there are four cases to consider:

1. $H$ and $C$ both true.
2. $H$ true and $C$ false.
3. $C$ true and $H$ false.
4. $H$ and $C$ both false.

**2.5 Proof by Contradiction:**

H and not C implies falsehood.

That is, start by assuming both the hypothesis $H$ and the negation of the conclusion $C$. Complete the proof by showing that something known to be false follows logically from $H$ and not $C$. This form of proof is called *proof by contradiction*.

It often is easier to prove that a statement is not a theorem than to prove it *is* a theorem. As we mentioned, if $S$ is any statement, then the statement "$S$ is not a theorem" is itself a statement without parameters, and thus can be regarded as an observation than a theorem.

**Alleged Theorem 1.13:** All primes are odd. (More formally, we might say: if integer $x$ is a prime, then $x$ is odd.)

**DISPROOF**: The integer 2 is a prime, but 2 is even. □

For any sets a,b,c if a ∩ b = Φ and c is a subset of b the prove that a ∩ c = Φ
Given : a ∩ b= Φ and c subset b
Assume: a ∩ c≠ Φ
Then ∀ x, xϵa and xϵc=>xϵb
= > a ∩ b ≠Φ = > a ∩ c= Φ (i.e., the assumption is wrong)

### 2.6 Proof by mathematical Induction:

Suppose we are given a statement $S(n)$, about an integer $n$, to prove. One common approach is to prove two things:

1. The *basis*, where we show $S(i)$ for a particular integer $i$. Usually, $i = 0$ or $i = 1$, but there are examples where we want to start at some higher $i$, perhaps because the statement $S$ is false for a few small integers.
2. The *inductive step*, where we assume $n \geq i$, where $i$ is the basis integer, and we show that "if $S(n)$ then $S(n+1)$."

- ***The Induction Principle***: If we prove $S(i)$ and we prove that for all $n \geq i$, $S(n)$ implies $S(n+1)$, then we may conclude $S(n)$ for all $n \geq i$.

## 3. Languages:

The languages we consider for our discussion is an abstraction of natural languages. That is, our focus here is on formal languages that need precise and formal definitions. Programming languages belong to this category.

### 3.1 Symbols:

Symbols are indivisible objects or entity that cannot be defined. That is, symbols are the atoms of the world of languages. A symbol is any single object such as , *a*, 0, 1, #, begin, or do.

### 3.2 Alphabets:

An alphabet is a finite, nonempty set of symbols. The alphabet of a language is normally denoted by Σ. When more than one alphabets are considered for discussion, then subscripts may be used (e.g. $\Sigma_1\Sigma_2$etc) or sometimes other symbol like G may also be

Σ = {0, 1}

Σ = {a, b, c}

Σ = {a, b, c, &, z}

Σ = {#, ▼, ♠, β}

### 3.3 Strings or Words over Alphabet:

A string or word over an alphabet Σ is a finite sequence of concatenated symbols of Σ.

**Example**: 0110, 11, 001 are three strings over the binary alphabet { 0, 1 } .

aab, abcb, b, cc are four strings over the alphabet { a, b, c }.

It is not the case that a string over some alphabet should contain all the symbols from the alphabet.

For example, the string cc over the alphabet { a, b, c } does not contain the symbols a and b.

Hence, it is true that a string over an alphabet is also a string over any superset of that alphabet.

### 3.4 Length of a string:

The number of symbols in a string w is called its length, denoted by |w|.

**Example:** | 011 | = 3, |11| = 2, | b | = 1, | 1001101 | = 7, | Ɛ | = 0,

For any symbol *c* and string *s*, we define the function ***#c***(*s*) to be the number of times that the symbol *c* occurs in *s*. So, for example, #a(abbaaa) = 4.

The most basic example of a string function is the length (string) function, which returns the length of a string (not counting any terminator characters or any of the string's internal structural information) and does not modify the string. For example, length ("helloworld") returns 11.

### 3.5 Some String Operations:

Let x and y be two strings. The concatenation of x and y denoted by xy, is the string. That is, the concatenation of x and y denoted by xy is the string that has a copy of x followed by a copy of y without any intervening space between them.

**Example:** Consider the string 011 over the binary alphabet. All the prefixes, suffixes and substrings of this string are listed below.

Prefixes: ε, 0, 01, 011.

Suffixes: ε, 1, 11, 011.

Substrings: ε, 0, 1, 01, 11, 011.

Note that x is a prefix (suffix or substring) to x, for any string x and ε is a prefix (suffix or substring) to any string.

A string x is a proper prefix (suffix) of string y if x is a prefix (suffix) of y and $x \neq y$.

In the above example, all prefixes except 011 are proper prefixes.

### 3.6 Powers of Strings:

For any string x and integer n>=0, we use $x^n$ to denote the string formed by sequentially concatenating n copies of x.

We can also give an inductive definition of as follows:

$x^n$ = e, if n = 0 ; otherwise

$x^n = x\ x^{n-1}$

**Example:** If $x = 011$, then $x^3 = 011011011$, $x^1 = 011$ and $x^0$**=e**

### 3.7 Relations on Strings

A string *s* is a *substring* of a string *t* iff *s* occurs contiguously as part of *t*. For example:

aaa is a substring of aaabbbaaa

aaaaaa is not a substring of aaabbbaaa

A string *s* is a *proper substring* of a string *t* iff *s* is a substring of *t* and $s \neq t$.

Every string is a substring (although not a proper substring) of itself. The empty string, ε, is a substring of every string.

### 3.8 Powers of Alphabets:

We write $\Sigma^k$ (for some integer k) to denote the set of strings of length k with symbols From $\Sigma$. In other words,

$\Sigma^k$= { w | w is a string over $\Sigma$ and | w | = k}. Hence, for any alphabet, $\Sigma^0$ denotes the set of all strings of length zero. That is, $\Sigma^0$= { e }.

For the binary alphabet { 0, 1 } we have the following.

$\Sigma^0$= {e}

$\Sigma^1$= {0,1}

$\Sigma^2$= {00, 01, 10, 11}

$\Sigma^3$= {000, 001, 010, 011, 100, 101, 110, 111}

The set of all strings over an alphabet $\Sigma$ is denoted by $\Sigma^*$ . That is,

$\Sigma^* = \Sigma^0 U \Sigma^1 U^2 U \ldots \Sigma^n U \ldots$

$= U\Sigma^k$.

The set $\Sigma^*$ contains all the strings that can be generated by iteratively concatenating symbols from $\Sigma$ any number of times.

**Example :** If = { a, b }, then $\Sigma^*$ = { ε, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, …}.

Please note that if $\Sigma=F$ , then $\Sigma^*$ that is $\Phi^*=\{e\}$. It may look odd that one can proceed from the empty set to a non-empty set by iterated concatenation.

### 3.9 Reversal:

For any string $w^n = a^1a^2a^3\ldots.a^n$ the reversal of the string is $w^n = aa^{n-1}\ldots.a^3a^2a^1$.

If *w* and *x* are strings, then $(w\,x)^R = x^R\,w^R$. For example, $(\text{nametag})^R = (\text{tag})^R\,(\text{name})^R$ = gateman

**Languages :**

A language over an alphabet is a set of strings over that alphabet. Therefore, a language L is any subset of $\Sigma^*$ . That is, any $L \subset \Sigma^*$is a language.

**Example :**

1. F is the empty language.
2. Σ*is a language for any Σ.
3. {e} is a language for any Σ. Note that, Φ≠{e} . Because the language F does not contain any string but {e} contains one string of length zero.
4. The set of all strings over { 0, 1 } containing equal number of 0's and 1's.
5. The set of all strings over {a, b, c} that starts with a.

**Convention :** Capital letters A, B, C, L, etc. with or without subscripts are normally used to denote languages.

**Set operations on languages :** Since languages are set of strings we can apply set operations to languages. Here are some simple examples (though there is nothing new in it).

**Union :**

A string $x \in L_1 \cup L_2$ iff $x \in L_1$ or $x \in L_2$

**Example :** { 0, 11, 01, 011 } U { 1, 01, 110 } = { 0, 1,11, 01, 011, 110 }

**Intersection :** A string, $x \in L_1 \cap L_2$ iff $x \in L_1$ and $x \in L_2$ .

**Example :** { 0, 11, 01, 011 } { 1, 01, 110 } = { 01 }

**Complement :** Usually, Σ* is the universe that a complement is taken with respect to. Thus for a language L, the complement is L(bar) = $\{x \in \Sigma^* \mid x \notin L\}$.

**Example :** Let L = { x | |x| is even }. Then its complement is the language { | |*x*| is odd }.

**Reversal of a language :**

The reversal of a language $L$, denoted as $L^R$, is defined as: $L^R = \{w^R | w \in L\}$

**Example :**

1. Let L = { 0, 11, 01, 011}. Then $L^R$= { 0, 11, 10, 110 }.

**Language concatenation :** The concatenation of languages L1 and L2 is defined as

$L_1L_2$= { $xy$ | x ∈ $L_1$ and y ∈ $L_2$ }.

**Example :** { *a*, *ab* }{ *b*, *ba* } = { *ab*, *aba*, *abb*, *abba* }.

Note that ,

1. $L_1L_2 \neq L_2L_1$ in general.
2. $L\Phi=\Phi$
3. $L\{e\}=L=\{e\}$

### 3.10 Iterated concatenation of languages :

Since we can concatenate two languages, we also repeat this to concatenate any number of languages. Or we can concatenate a language with itself any number of times. The operation $L^n$ denotes the concatenation of L with itself n times. This is defined formally as follows:

$L_0=\{e\}$

$L^n=LL^{n-1}$

**Example :** Let L = { a, ab }. Then according to the definition, we have

$L_0=\{e\}$

$L_1=L\{e\}=L=\{a, ab\}$

$L_2=LL_1=\{a, ab\}\{a, ab\}=\{aa, aab,aba,abab\}$

$L_3= LL_1=\{a, ab\}\{aa, aab,aba,abab\} = \{aaa, aaab, aaba, aabab, abaa, abaab, ababa, ababab\}$

and so on.

## 4. Automata theory

In theoretical computer science, automata theory is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.

This automaton consists of

• states (represented in the figure by circles),

• and transitions (represented by arrows).

As the automaton sees a symbol of input, it makes a *transition* (or *jump*) to another state, according to its *transition function* (which takes the current state and the recent symbol as its inputs). e.g.

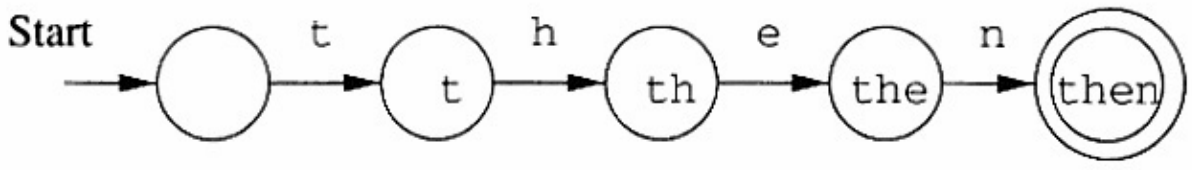
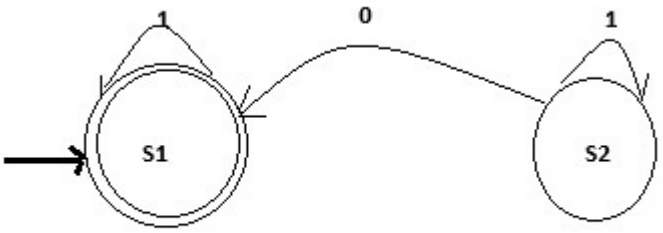



Figure : A finite automaton modeling recognition of **then**

Another example of automata is as follows:




Sample of Automata

The Figure above illustrates a finite state machine, which is one well-known variety of automata. This automaton consists of states (represented in the figure by circles), and transitions (represented by arrows). As the automaton sees a symbol of input, it makes a *transition* (or *jump*) to another state, according to its *transition function* (which takes the current state and the recent symbol as its inputs).

Automata theory is also closely related to formal language theory, as the automata are often classified by the class of formal languages they are able to recognize. An automaton can be a finite representation of a formal language that may be an infinite set.

In other words, automata theory is a subject matter which studies properties of various types of automata. For example, following questions are studied about a given type of automata.

i. Which class of formal languages is recognizable by some type of automata? (Recognizable languages)
ii. Is certain automata *closed* under union, intersection, or complementation of formal languages? (Closure properties)
iii. How much is a type of automata expressive in terms of recognizing class of formal languages? And, their relative expressive power? (Language Hierarchy)
iv. Automata theory also studies if there exist any effective algorithm or not to solve problems similar to the following list:
v. Does an automaton accept any input word? (emptiness checking)
vi. Is it possible to transform a given non-deterministic automaton into deterministic automaton without changing the recognizing language? (Determinization)
vii. For a given formal language, what is the smallest automaton that recognizes it? (Minimization).

### 4.1 Applications of Automata Theory

Each model in automata theory play varied roles in several applied areas. Finite automata are used in text processing, compilers, and hardware design. Context-free grammar (CFG) is used in programming languages and artificial intelligence. Originally, CFG were used in the study of the human languages. Cellular automata are used in the field of biology, the most common example being John Conway's Game of Life. Some other examples which could be explained using automata theory in biology include mollusk and pinecones growth and pigmentation patterns. Going further, Stephen Wolfram claims that the entire universe could be explained by machines with a finite set of states and rules with a single initial condition. Other areas of interest which he

has related to automata theory include: fluid flow, snowflake and crystal formation, chaos theory, cosmology, and financial analysis. Automata play a major role in compiler design and parsing.

**Automata** can also be define as an algorithm or program that automatically recognizes if a particular string belongs to the language or not, by checking the grammar of the string.

Every Automaton fulfills the following basic requirements.

• Every automaton consists of some essential features as in real computers. It has a mechanism for reading input. The input is assumed to be a sequence of symbols over a given alphabet and is placed on an input tape (or written on an input file). The simpler automata can only read the input one symbol at a time from left to right but not change. Powerful versions can both read (from left to right or right to left) and change the input.

•. The automaton can produce output of some form. If the output in response to an input string is binary (say, accept or reject), then it is called an accepter. If it produces an output sequence in response to an input sequence, then it is called a transducer (or automaton with output).

• The automaton may have a temporary storage, consisting of an unlimited number of cells, each capable of holding a symbol from an alphabet (whcih may be different from the input alphabet). The automaton can both read and change the contents of the storage cells in the temporary storage. The accusing capability of this storage varies depending on the type of the storage.

• The most important feature of the automaton is its control unit, which can be in any one of a finite number of interval states at any point. It can change state in some defined manner determined by a transition function.

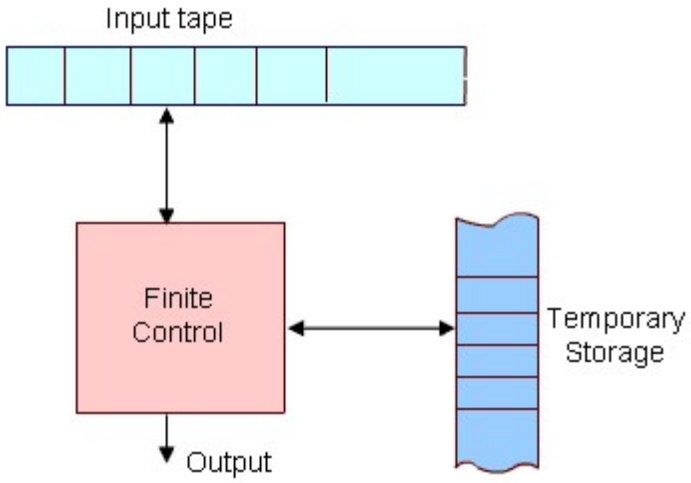


Figure: The figure above shows a diagrammatic representation of a generic automation.

## 4.2 Finite Automata/ Finite State Automata (FSA)

Automata (singular: automation) are a particularly simple, but useful, model of computation. They were initially proposed as a simple model for the behavior of neurons.

**FSA** is characterized by not having a temporary storage compare to pushdown automata or Turing Machines. The FSAs have a little capacity to remember its computation since its input cannot be rewritten into memory. Thus, the term **"finite"** means an explicit amount of information can be retained in the Control Unit by placing the Unit into a specific state. But since the number of such states is finite, a finite automaton can only deal with situations in which the information to be stored at any time is strictly bounded

The finite-state automata (FSA) or finite state machine (FSM) enjoy a special place in computer science. The FSA has proven to be a very useful model for many practical tasks and deserves to be among the tools of every practicing computer scientist.

**States, Transitions and Finite-State Transition System:**

Let us first give some intuitive idea about *a state of a system* and *state transitions* before describing finite automata.

Informally, *a state of a system* is an instantaneous description of that system which gives all relevant information necessary to determine how the system can evolve from that point on.

*Transitions* are changes of states that can occur spontaneously or in response to inputs to the states. Though transitions usually take time, we assume that state transitions are instantaneous (which is an abstraction).

Some examples of state transition systems are: digital systems, vending machines, etc. A system containing only a finite number of states and transitions among them is called a *finite-state transition system*.

Finite-state transition systems can be modeled abstractly by a mathematical model called *finite automation*

The figures above are examples of finite state automata.

**Classes of automata**

In the following table is an incomplete list of some types of automata.

| **Automata** | **Recognizable language** |
|---|---|
| Deterministic finite automata (DFA) | regular languages |
| Nondeterministic finite automata (NFA) | regular languages |
| Nondeterministic finite automata with ε transitions (FND-ε or ε-NFA) | regular languages |
| Pushdown automata (PDA) | context-free languages |
| Linear bounded automata (LBA) | context-sensitive language |
| Turing machines | Recursively enumerable languages, etc. |

## 4.3 Deterministic Finite (-state) Automata (DFA)

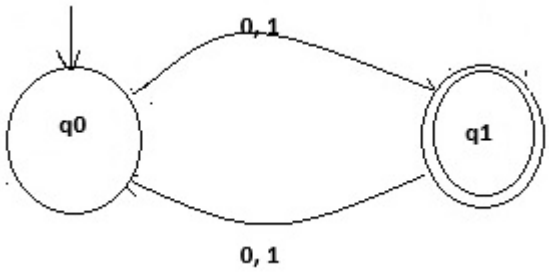
i. DFAs are:
ii. Deterministic i.e. there is no element of choice
iii. Finite i.e. only a finite number of states and arcs
iv. Acceptors i.e. produce only a yes/no answer

A DFA is drawn as a graph, with each *state* represented by a circle.

Informally, a DFA (Deterministic Finite State Automaton) is a simple machine that reads an input string -- one symbol at a time -- and then, after the input has been completely read, decides whether to accept or reject the input. As the symbols are read from the tape, the automaton can change its state, to reflect how it reacts to what it has seen so far. A machine for which a

deterministic code can be formulated, and if there is only one unique way to formulate the code, then the machine is called deterministic finite automata. Example:

These machines will have a collection of \states", and each time they process a character they will “transition" to a new state. Here is a picture of a machine that recognizes strings of odd length:




When processing a string, the machine starts in state q0 (as indicated by the arrow pointing at q0. When it is in a given state, it reads a character from the input, and follows the edge corresponding to that character from the state it is in to find a new state It will say “yes" on the string x if it is in the accept state (designated by the double circle) after processing x completely. If we run this machine on the input “010", it will start in q0. It will read ”0" which causes it to transition to q1. It will read “1", causing it to transition back to q0. Finally, it will read \0", and transition back to q1. Since q1 is an accept state, it will say “yes".

## Formalism

We can encode all of the parts of the picture above symbolically as follows. A machine

M consists of the following:

_ A finite set Q of states ({q0; q1} in the example)

_ An initial state q0 $\epsilon$Q. (q0 in the example)

_ A set of “accepting states" A $\subset$Q ({q1} in the example)

_ A transition function $\delta$: QX$\Sigma$→ Q. In the example, $\delta$(q0; 0) = $\delta$(q0; 1) =

q1 and $\delta$(q1; 0) = _(q1; 1) = q0.

Using this encoding, we can give a crisp definition of the informal process described above. We can use $\delta$ to define a function $\delta$: QX$\Sigma$*→ Q that gives the state of the machine after processing an entire string. For the machine above, $\delta$(010) = q1.

$\delta$ is defined inductively. For any state q,

_ $\delta(q;\epsilon) = q$

_ $\delta(q; ax) = \delta(\delta(q; a); x)$

Then M accepts x if $\delta(q0; x) \epsilon A$.

**Algorithm for the Operation of a DFA**

 Start with the "current state" set to the start state and a "read head" at the beginning of the input string;

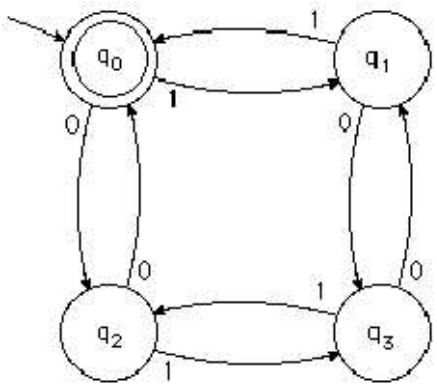
 while there are still characters in the string:

o Read the next character and advance the read head;

o From the current state, follow the arc that is labelled with the character just read; the state that the arc points to becomes the next current state;

 When all characters have been read, *accept* the string if the current state is a final state, otherwise *reject* the string.

Example 2:




***Example 2 of DFA***

Consider the following input string: 1 0 0 1 1 1 0 0. Using the DFA in Figure above, a sample trace will be as follows:

q0 1 q1 0 q3 0 q1 1 q0 1 q1 1 q0 0 q2 0 q0. Since q0 is a final state, the string is accepted.

**Implementing above DFA**

**Using a GO TO Statement**

If you do not object to the **go to** statement, below is an easy way to implement a DFA:

```
q0 : read char;
if eof then accept string;
if char = 0 then go to q2;
if char = 1 then go to q1;

q1 : read char;
if eof then reject string;
if char = 0 then go to q3;
if char = 1 then go to q0;

q2 : read char;
if eof then reject string;
if char = 0 then go to q0;
if char = 1 then go to q3;

q3 : read char;
if eof then reject string;
if char = 0 then go to q1;
if char = 1 then go to q2;
```

### Using a CASE Statement

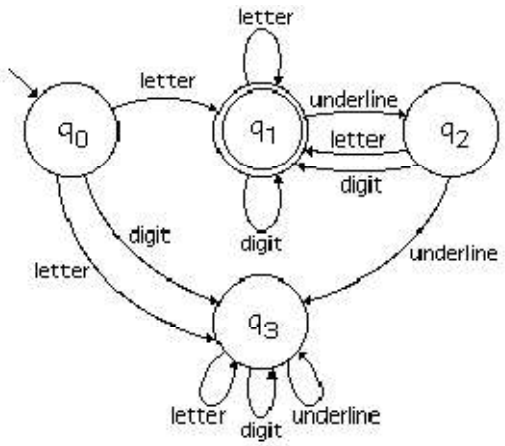
If you are not allowed to use a **go to** statement, you can as well use a combination of a loop and a case statement:

```
state := q0;
loop
case state of

q0 : read char;
if eof then accept string;
if char = 0 then state := q2;
if char = 1 then state := q1;

q1 : read char;
if eof then reject string;
if char = 0 then state := q3;
if char = 1 then state := q0;

q2 : read char;
if eof then reject string;
if char = 0 then state := q0;
if char = 1 then state := q3;

q3 : read char;
if eof then reject string;
if char = 0 then state := q1;
if char = 1 then state := q2;
end case;
end loop;
```

**Acceptor for Ada identifiers**

In Ada, an identifier consists of a letter followed by any number of letters, digits, and underlines. However, the identifier may not end in an underline or have two underlines in a row.

Here is an automaton to recognize Ada identifiers.




Formal acceptor for Ada identifiers.

## 4.4 Nondeterministic finite automata (NFA)

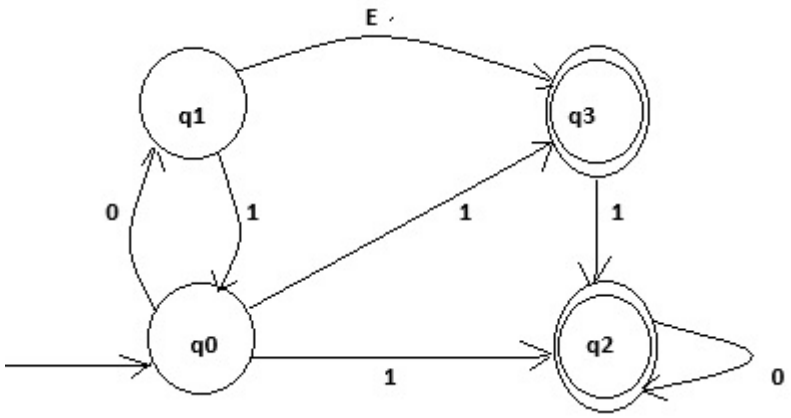
An FSA is nondeterministic if it is confronted with several choices when processing each character. Thus the transition function of a nondeterministic FSA maps state/input pairs into sets of states. The machine somehow traverses all possible paths in parallel. In addition, E transitions are permitted, allowing the machine to change states without reading an input character. A string is accepted by an NFA if one of its parallel transition sequences leads to a final state.

It is also useful to loosen the restrictions on building machines to allow multiple transitions on the same character, or no transitions at all. This allows us to more easily express some constructions, but turns out not to add any computational power (as we will see later).

We will also add “ϵ transitions," allowing the machine to transition from one state to another without consuming any input.

A machine will accept the input x if there is ANY path from the start state to an accepting state that is labeled by x.

Here is an example NFA:




On the input 010, this machine will operate as follows. It begins in state $q_0$.

After processing the first 0, it can transition to state $q_1$, and it can also take the ϵ transition to state $q_3$. So $\hat{\delta}(q_0; 0) = \{q_1; q_3\}$.

It will then process the 1, allowing it to transition to state $q_0$ (from state $q_1$) or to state $q_2$ (from state $q_3$). Finally it will process the second 0, allowing it to end in state $q_1$, $q_2$, or $q_3$. Since at least one of these is an accept state, the string 010 will be accepted.

Another way to look at the process is that we can insert some ϵs into the string 010 so that it gives a path from the start state to the accepting state: we can expand it to 0ϵ10, and take the path $q_0 \rightarrow q_1 \rightarrow q_3 \rightarrow q_2 \rightarrow q_2$. Since $q_2$ is an accepting state, we accept.

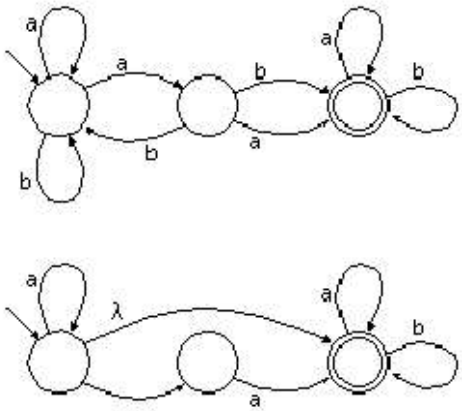



Fig. ***Nondeterministic Finite Acceptor***

A state may have two or more arcs emanating from it labelled with the same symbol. When the symbol occurs in the input, either arc may be followed. A state may have one or more arcs emanating from it labelled with ג (the empty string) . These arcs may optionally be followed without looking at the input or consuming an input symbol.

Due to nondeterminism, the same string may cause an NFA to end up in one of several different states, some of which may be final while others are not. The string is accepted if **any** possible ending state is a final state. Other examples are as follows:

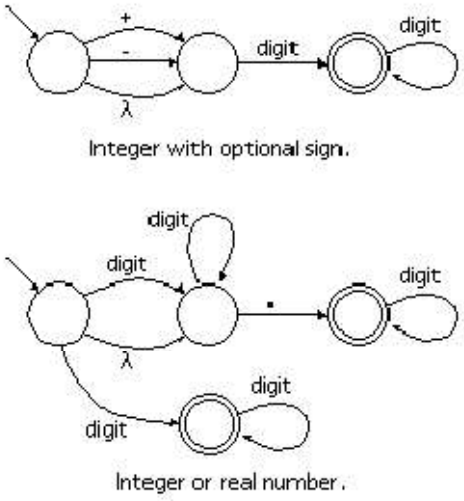



Fig. ***Examples of NFAs***

**Implementing an NFA**

If you think of an automaton as a computer, how does it handle nondeterminism?

There are three ways, two feasible and one not yet feasible, to simulate the second alternative:

1. Use a recursive backtracking algorithm. Whenever the automaton has to make a choice, cycle through all the alternatives and make a recursive call to determine whether any of the alternatives leads to a solution (final state).

2. Maintain a state set or a state vector, keeping track of *all* the states that the NFA could be in at any given point in the string.

3. Use a *quantum computer*. Quantum computers explore literally all possibilities simultaneously. They are theoretically possible, but are at the cutting edge of physics. It may (or may not) be feasible to build such a device.

## 4.5 Pushdown Automata

The finite-state automaton (FSA) and the pushdown automaton (PDA) enjoy a special place in computer science. The FSA has proven to be a very useful model for many practical tasks and deserves to be among the tools of every practicing computer scientist. Many simple tasks, such

as interpreting the commands typed into a keyboard or running a calculator, can be modelled by finite-state machines.

The PDA is a model to which one appeals when writing compilers because it captures the essential architectural features needed to parse context-free languages, languages whose structure most closely resembles that of many programming languages.

A DFA (or NFA) is not powerful enough to recognize many context-free languages because a DFA cannot count. But counting is not enough.

Consider a language of palindromes, containing strings of the form wwR. Such a language requires more than an ability to count; it requires a stack.

A *pushdown automaton (PDA)* is basically an NFA with a stack added to it.

This machine is fed input just as a finite automaton is. Some texts speak of the input coming in on a read-only tape with a tape head that moves left to right until it comes to the end of the input. At that point it reads a special character that marks a blank tape cell. We will use the character Δ or ג as a "blank". When the tape head reads a blank the machine halts, or begins the process of halting.

The tape head may not reverse directions, nor may it be used to write to the tape. The input tape is infinitely long in the rightward direction, which allows a PDA to accept a finite input of any length. The cells of the tape are numbered, with the first input character occurring in cell 1 where the tape head is set initially.

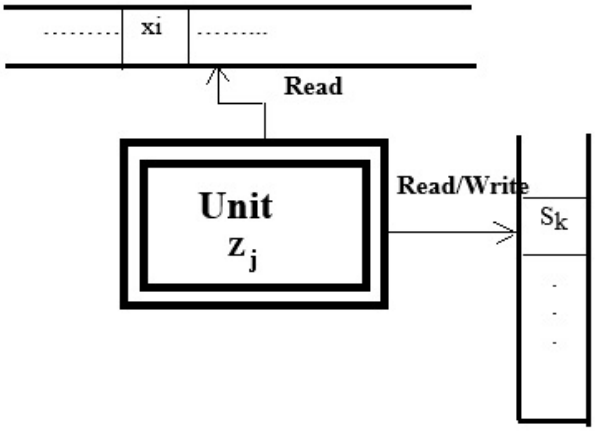
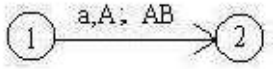



**Figure :** Conceptual Model of a Pushdown Automaton

In addition to the input tape, a PDA has an associated stack onto which it can push characters to remember them. This stack has no limit to its size so the PDA can push as many characters as it likes. The machine begins processing with an empty stack. Typically the first thing the machine does is push a "bottom-of-the-stack marker" onto the stack. We shall use the Δ as that marker. Note that a PDA has two associated alphabets, one containing characters that may appear on the input tape, the other containing characters that may be pushed onto the stack.

For example, suppose we find the following transition in a PDA:




This transition says that if we are in state 1 and there is an **a** in the current cell of the input tape and we pop an **A** off the top of the stack, then we may go to state 2 and must push the string **AB** onto the stack, pushing the **A** first and then the **B**. We may use a Δ in any of the three parts of the transition label. It always means that we do not do the task that part of the label involves. In other words, a Δ in place of an input character means that we do not read from the tape. A Δ in place of the character popped off the stack means that we do not pop anything off the stack, and a Δ in place of the string to be pushed means that we do not push anything.

Here is a machine that accepts the language $\{a^n b^n \mid n \geq 0\}$. The machine begins with its input on the input tape and an empty stack. The first thing the machine does is go from state 0 to state 1, pushing the bottom-of-the-stack marker onto the stack.

In state 1, if the machine reads an **a** from the input and pops the blank off the stack, then that is the first a found in the input and the machine pushes the blank back onto the stack followed by an **A**. The machine counts the **a**'s in the input by pushing an **A** onto the stack for each **a** that it reads off the tape. From this point on, if the PDA is in state 1 and it finds an **a** in the input, there will be an **A** to pop off the stack. The machine then pushes two **A**'s back on the stack, one to make up for the **A** that was popped and one to count the **a** just found in the input. As soon as the machine sees a **b** in the input it changes to state 2 and pops an **A** off the stack. It continues popping an **A** for each **b** that it finds. If the input was a correctly formatted string, the machine will read a blank off the input tape at the same time that it pops a blank off the stack and go to state

3, an accept state. Since the language includes the empty string, we also have a transition from state 1 to state 3 that is used if the machine reads a blank from the input at the same time as it pops the marker off the stack at the very beginning of processing.

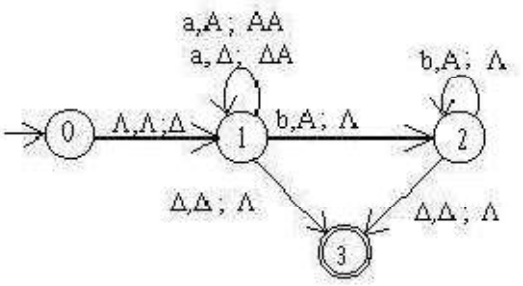



**Figure: Machine that accepts the language** $\{a^n b^n \mid n \geq 0\}$

https://www.youtube.com/watch?v=ntrF_KxHn18

The previous machine shows that PDAs have more power than FSAs because that machine accepts a nonregular language, something that an FSA cannot do.

**Types of PDAs**

Like FSA, there are two types of PDAs:

1. A *deterministic PDA* (DPDA) is one in which every input string has a unique path through the machine.
2. A *nondeterministic PDA* (NPDA) is one in which we may have to choose among several paths for an input string.

We say that an input string is accepted if there is at least one path that leads to an accept state. Nondeterministic PDA is more powerful than a deterministic one, unlike the situation with DFAs and NFAs. In other words, there are languages that can be accepted by an NPDA that cannot be accepted by a DPDA. ETC.

# 5 Formal Grammars,

## 5.1 Grammar

A grammar is a mechanism used for describing languages. In everyday language, like English, we have a set of symbols (alphabet), a set of words constructed from these symbols, and a set of rules using which we can group the words to construct meaningful sentences. The grammar for English tells us what are the words in it and the rules to construct sentences. It also tells us whether a particular sentence is well-formed (as per the

grammar) or not. But even if one follows the rules of the english grammar it may lead to some sentences which are not meaningful at all, because of impreciseness and ambiguities involved in the language. In english grammar we use many other higher level constructs like noun-phrase, verb-phrase, article, noun, predicate, verb etc. A typical rule can be defined as

< sentence >→ < noun-phrase > < predicate >

meaning that "a sentence can be constructed using a 'noun-phrase' followed by a predicate".

Some more rules are as follows:

< noun-phrase >→ < article >< noun >

< predicate > →< verb >

with similar kind of interpretation given above.

If we take {a, an, the} to be <article>; cow, bird, boy, Ram, pen to be examples of <noun>; and eats, runs, swims, walks, are associated with <verb>, then we can construct the sentence- a cow runs, the boy eats, an pen walks- using the above rules. Even though all sentences are well-formed, the last one is not meaningful.

We observe that we start with the higher level construct <sentence> and then reduce it to <noun-phrase>, <article>, <noun>, <verb> successively, eventually leading to a group of words associated with these constructs.

These concepts are generalized in **formal language** leading to **formal grammars**. The word 'formal' here refers to the fact that the specified rules for the language are explicitly stated in terms of what strings or symbols can occur. There can be **no ambiguity** in it.

Formal definitions of a Grammar

### 5.2 Definition of Formal Grammar

A **formal grammar** (sometimes simply called a **grammar**) is a set of rules for forming strings in a formal language. The rules describe how to form strings from the language's alphabet that are valid according to the language's syntax. A grammar does not describe the meaning of the strings or what can be done with them in whatever context – only their form.

A formal grammar can also be defined as a precise **mathematical system of symbols and production rules** that defines all possible strings (syntax) in a formal language. It acts as language generator and backbone for compilers, parsers and natural language processing or a rulebook to generate valid strings and a foundation for compilers to verify syntax and parse code.

## Formal definitions of a Grammar:

Formally, a grammar is defined as a mathematical 4-tuple:

A grammar $G$ is defined as a quadruple.

$G = (N, T, P, S)$

***N*(Non-terminals):** Symbols that can be replaced or expanded by production rules (e.g., variables, sentence parts). Or N is a non-empty finite set of non-terminals or variables,

**T(Terminals) or Σ:** The basic, unchangeable symbols or characters that make up the final valid string. Or is a non-empty finite set of terminal symbols

***P*(Production rules):** The rules dictating how non-terminals can be replaced by other symbols. A→aB means A can be replaced by aB. (e.g .A→aB).

***S* (Start Symbol):** A specific non-terminal from which all string derivations must begin.

**e.g.**

**SϵN** is a special non-terminal (or variable) called the start symbol, and $P \subset (NU\Sigma)^+ \times (NU\Sigma)^*$ is a finite set of production rules.

The binary relation defined by the set of production rules is denoted by →, i.e. a → β iff (a, β) ϵ P .

In other words, P is a finite set of production rules of the form a → β, where a ϵ $(NU\Sigma)^+$ and β ϵ $(NU\Sigma)^*$

## 5.3 Production rules:

The production rules specify how the grammar transforms one string to another. Given a string δαy, we say that the production rule α→β is applicable to this string, since it is possible to use the rule α→β to rewrite the α (in δαy) to β obtaining a new string δβy. We say that δαy derives δβy and is denoted as

δαy ➔ δβy

Successive strings are dervied by applying the productions rules of the grammar in any arbitrary order. A particular rule can be used if it is applicable, and it can be applied as many times as described.

### 5.4 Examples

**Example 1:** Consider the grammar G=(N, T, P, S), where $N$ = {$S$}, P={$a$, $b$} and P is the set of the following production rules

$$\{ S \rightarrow ab,\ S \rightarrow aSb \}$$

Some terminal strings generated by this grammar together with their derivation is given below.

S➔ ab

S ➔aSb ➔aabb

S ➔aSb ➔aaSbb ➔aaabbb

It is easy to prove that the language generated by this grammar is

$L(G) = \{a^i b^i \mid i \geq 1\}$

By using the first production, it generates the string ab ( for $i$ =1 ).

To generate any other string, it needs to start with the production $S \rightarrow aSb$ and then the non-terminal $S$ in the

RHS can be replaced either by *ab* (*in which we get the string aabb*) or the same production $S \rightarrow aSb$ can be used one or more times. Every time it adds an '*a*' to the left and a '*b*' to the right of S, thus giving the sentential form $a^iSb^i$ , $i \geq 1$. When the non-terminal is replaced by *ab* (which is then only possibility for generating a terminal string) we get a terminal string of the form $a^ib^i$ , $i \geq 1$. This is equivalent to $a^nb^n$ , $n \geq 1$

**Example 2:** Consider the grammar *G* where N={S, B},Σ={a, b, c} , *S* is the start symbol, and *P* consists of the following production rules:

1. S→aBSc
2. S→abc
3. Ba→aB
4. Bb→bb

This grammar defines the language

$$L(G) = \{a^n b^n c^n | n \geq 1\}$$

where $a^n$ denotes a string of *n* consecutive *a*'s. Thus, the language is the set of strings that consist of 1 or more *a*'s, followed by the same number of *b*'s, followed by the same number of *c*'s.

Some examples of the derivation of strings in *L*(*G*) are:

$$\boldsymbol{S} \Rightarrow_2 \boldsymbol{abc}$$

$$\boldsymbol{S} \Rightarrow_1 \boldsymbol{aBSc} \Rightarrow_2 aB\boldsymbol{abc}c \Rightarrow_3 a\boldsymbol{a}B\boldsymbol{b}cc \Rightarrow_4 aa\boldsymbol{bb}cc$$

$$\boldsymbol{S} \Rightarrow_1 \boldsymbol{aBSc} \Rightarrow_1 aB\boldsymbol{aBSc}c \Rightarrow_2 aBaB\boldsymbol{abc}cc \Rightarrow_3 a\boldsymbol{a}B\boldsymbol{B}abccc \Rightarrow_3 aaB\boldsymbol{a}\boldsymbol{B}bcc$$

$$\Rightarrow_3 aa\boldsymbol{a}\boldsymbol{B}Bbccc \Rightarrow_4 aaaB\boldsymbol{bb}ccc \Rightarrow_4 aaa\boldsymbol{bb}bccc$$

**Equivalent to** $L(G) = \{a^3 b^3 c^3 | n=3\}$

## 5.5 The Chomsky Hierarchy

When Noam Chomsky first formalized generative grammars in 1956, he classified them into types now known as the Chomsky hierarchy. The difference between these types is that they have increasingly strict production rules and can express fewer formal languages.

Formal grammars are classified into four hierarchical types, depending on how strict their production rules are:

1. **Type 0 (Unrestricted):** The most general grammar; production rules can take any form.
2. **Type 1 (Context-Sensitive):** Rules apply only within the context of specific surrounding symbols.
3. **Type 2 (Context-Free):** The left side of a production rule must be a single non-terminal, regardless of context.
4. **Type 3 (Regular):** The strictest grammar, used for pattern matching and simple token recognition.

Two important types are ***context-free grammars*** **(Type 2)** and ***regular grammars*** **(Type 3)**. The languages that can be described with such a grammar are called *context-free languages* and *regular languages*, respectively. Although much **less powerful** than unrestricted grammars (Type 0), which can in fact express any language that can be accepted by a Turing machine, these two restricted types of grammars are most often used because parsers for them can be efficiently implemented.

For example, all regular languages can be recognized by a finite state machine, and for useful subsets of context-free grammars there are well-known algorithms to generate efficient LL parsers and LR parsers to recognize the corresponding languages those grammars generate.

## Chomsky Grammars:

Chomsky has provided ways of defining numbers (digits) and letters which are useful in designing transition diagrams for a scanner. There are four types of this grammar:

**TYPE 0**

Unrestricted grammar:

This is of the form:

Q ———> B

There is no restriction on Q or B, both of which are called sentential forms. This grammar is too general for computer languages.

**TYPE 1**

Context Sensitive grammar:

This has a production of the form:

Q A B ————> QπB.

A is a single Non-Terminal (NT) symbol and is not null.

Q, B, π are sentential forms. Language in this class are said to be Context sensitive.

Production on the right hand side depends on the symbols on the LHS and their arrangement restrictions on them.

**Example**

S ————> B A d

B ————> b/c

A ————> Z

bAd————> bAd

**Type 2**

Context-free grammar:

The production is of the form:

A ————> π

Here the LHS consists of one NT symbol. Most programming languages are of this form. However, languages which do not correspond with this form can cause inconsistencies but are only minor.

**Example:**

The language defined above is not a context-free language, and this can be strictly proven using the pumping lemma for context-free languages, but for example the language $L(G) = \{a^n b^n | n \geq 1\}$ (at least 1 *a* followed by the same number of *b*'s) is context-free, as it can be defined by the grammar $G_2$ with $N = \{S\}$, $\Sigma = \{a, b\}$, $S$ the start symbol, and the following production rules:

1. S→aSb
2. S→ab

**Type 3:**

Regular grammar:

The production is of the form:

A ————> aB

A ————> a

The RHS contain only terminal symbols or terminal symbol followed by a NT symbol. This type is needed at lexical scan stage and it coincides with finite state languages.

In regular grammars, the left hand side is again only a single nonterminal symbol, but now the right-hand side is also restricted. The right side may be the empty string, or a single terminal symbol, or a single terminal symbol followed by a nonterminal symbol, but nothing else. (Sometimes a broader definition is used: one can allow longer strings of terminals or single nonterminals without anything else, making languages easier to denote while still defining the same class of languages.)

**Example:**

The language defined above is not regular, but the language $\{a^n b^m \mid m, n \geq 1\}$ is (at least 1 *a* followed by at least 1 *b*, where the numbers may be different), as it can be defined by the grammar $G_3$ with $N = \{S, A, B\}$, $\Sigma = \{a, b\}$, $S$ the start symbol, and the following production rules:

1. S→aA
2. A→aA
3. A→bB
4. B→bB
5. B→ϵ

All languages generated by a regular grammar can be recognized in linear time by a finite state machine.

## 6 Parsing

Parsing is the process of breaking down a complex stream of data or text into smaller, readable components based on a set of formal grammar rules. It converts raw, unstructured data into organized structures like trees or tables that humans or machines can interpret and execute.

A language is a set of strings, and any well-defined set must have a membership criterion. A context-free grammar can be used as a membership criterion – **if** we can find a general algorithm for using the grammar to recognize strings.

*Parsing* a string is finding a derivation (or a derivation tree) for that string.

Parsing a string is like recognizing a string. An algorithm to recognize a string will give us only a yes/no answer; an algorithm to parse a string will give us additional information about how the string can be formed from the grammar.

## 6.1 Primary Applications

i. **Computer Science & Compilers:** Translators and compilers break down programming code into an Abstract Syntax Tree (AST) so the machine can execute instructions. It is also how web browsers process HTML into the Document Object Model (DOM).
ii. **Natural Language Processing (NLP):** AI and linguistic systems parse human speech or text to understand syntax, define the relationship between words, and map out sentences in a parse tree.
iii. **Data Processing:** Systems parse raw formats like JSON, XML, or CSV into usable variables or databases.

## 6.2 Why is parsing important?

i. **Grammar Verification:** It ensures that words fit together logically, identifying errors like incorrect verb tenses, dangling modifiers, or subject-verb disagreements
ii. **Comprehension:** It allows readers or natural language processing (NLP) systems to break down complex, multi-clause sentences to extract accurate meaning.

## 6.3 How Parsers Work

Parsers generally use algorithms based on formal grammars to check input line by line. These rules dictate whether a sentence or string is valid.

i. **Top-down Parsing:** Begins at the highest level of the structure (e.g., the sentence root) and breaks it down into individual words.
ii. **Bottom-up Parsing:** Begins with the smallest components (the words) and combines them into higher-level logical structures.

## 6.4 How Parsing Works

When you parse a sentence, you typically perform two main steps:

1. **Identify the Parts of Speech:** Classify every single word (e.g., noun, verb, adjective, preposition).
2. **Find Syntactic Constituents:** Group words into logical phrases (e.g., Noun Phrases and Verb Phrases)
   For example, take the sentence: *"The cat chased the mouse."*

- **Noun Phrase (Subject):** "The cat"
- **Verb Phrase (Predicate):** "chased the mouse" (which is made of the verb "chased" + the Noun Phrase object "the mouse")

## 6.5 Common Methods

Parsing can be a hands-on exercise or a highly automated procedure

i. **Sentence Diagramming:** A traditional, visual method of drawing lines to connect the subject, verb, and modifiers to show how a sentence is built.

ii. **Parse Trees:** In linguistics and computer science, sentences are often represented as a branching tree diagram that illustrates the hierarchical structure of phrases.

iii. **Algorithmic Parsing:** This is the foundation of computer programming and compiler design, where software breaks down and understands formal programming languages using defined sets of rules

Parse Trees:

There is a tree representation for derivations that has proved extremely useful. This tree shows us clearly how the symbols of a terminal string are grouped into substrings, each of which belongs to the language of one of the variables of the grammar. But perhaps more importantly, the tree, known as a "parse tree"

**Example**:

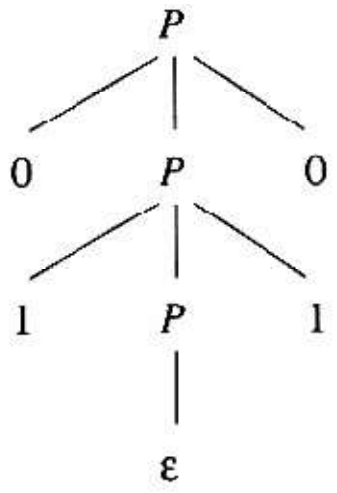



Figure 5.5: A parse tree showing the derivation $P \stackrel{*}{\Rightarrow} 0110$

## 7 Relative powers of formal models.

The "relative power" of formal models refers to their expressive capability, mathematical tractability, and capacity to represent complex systems. In computer science and mathematics, this is famously organized by the **Chomsky Hierarchy**, which scales from the most restricted (Finite State Automata) to the most powerful (Turing Machines).

The relative expressive power and capabilities of these foundational models vary significantly:

## 1. Finite State Automata (FSA) / Regular Languages

- **Power:** Lowest computational power.
- **Capabilities:** Recognizes regular expressions. Cannot count arbitrary things (e.g., it can't track whether the number of '(' and ')' is perfectly balanced in a string because it has no memory stack).
- **Use Cases:** Simple lexical analysis in compilers, vending machine operations, and basic text matching

## 2. Pushdown Automata (PDA) / Context-Free Languages

- **Power:** Moderate computational power.
- **Capabilities:** Adds a stack (Last-In, First-Out memory), enabling the model to track nested structures and balances (like matching parentheses or HTML tags).
- **Use Cases:** Parsing programming languages, syntax analysis, and basic natural language processing (NLP) structures

## 3. Linear Bounded Automata (LBA) / Context-Sensitive Languages

- **Power:** High computational power.
- **Capabilities:** Recognizes context-sensitive grammars. The memory scales linearly with the length of the input.
- **Use Cases:** Advanced syntactic analysis and formal representations where the meaning or legality of a symbol depends on its surrounding environment.

## 4. Turing Machines / Recursively Enumerable Languages

- **Power:** Universal computational power.
- **Capabilities:** Can simulate any computer algorithm. Features an infinite tape. A Universal Turing Machine can simulate *any* other Turing machine, making it the theoretical foundation of modern computing devices.
- **Use Cases:** Defining computability, solving algorithm complexity limits, and verifying arbitrary software systems using rigorous

## Additional Analytical Models

Outside of computer science, formal models span across scientific and sociopolitical disciplines, categorized by their distinct representations:

- **Mathematical & Logical Structures:** Used across disciplines to give precise meaning to arguments and enforce rigor in thought experiments.

- **Formal Models in Normative Political Theory:** Employed to make rigorous thought experiments and test the consistency of normative principles when empirical data is absent.
- **Data vs. Theory Models:** In the sciences, formal data models (patterned observations) are contrasted with theory models (processes explaining the observations). Scientific explanation is typically achieved when these two models are isomorphic.
- **Game Theory & Conflict:** Used to structure the characteristics of a dispute, analyzing choices, available courses of action, and outcomes based on decision-maker preferences

## 8 Basic computability:

The answer to the computability question. Let's say that a "program" is just a string containing program (e.g. Java source code) code for a function that takes a string as input and returns a Boolean (yes/no true/false, 1/0). Then any string that is not valid Java source code as a program that says "no" on every input-thus every string can be interpreted as a program.

We can now ask if every language L has a program P that recognises it.

The set of languages is just the set of sets of strings:

$$L=P(E^*)$$

The set of programs is the set of strings:

$$P=E^*$$

We have the function L(.) that takes a program and gives that language of the program:

$$L(.):E^* \rightarrow P(E^*)$$

Asking whether every specification has a program is the same as asking if L(.) is onto where every element in the target set has at least one matching element in the starting set.

The answer is clearly no, because $|P(E^*)|>|E^*|$, so there are no onto function from $E^*$ to $P(E^*)$.

## 9 Turing-Machines,

A Turing machine is a foundational theoretical model of computation invented by mathematician Alan Turing in 1936. It represents an idealized, abstract version of a computer CPU. Despite its extreme simplicity, it can simulate the logic of any computer algorithm.

The Turing machine is not a physical device, but a mathematical concept. It consists of three primary components:

- **An Infinite Tape:** Divided into sequential cells, each containing a symbol (such as \(1\), \(0\), or a blank space). This acts as the machine's memory, which can be read, written to, or erased.
- **A Tape Head:** Acts as a pointer that reads the symbol on the current cell, edits the symbol, and moves the tape one step to the left or right.
- **A State Machine (Program):** A finite list of rules that dictate what the machine should do. The action taken depends on two things: the machine's current internal state and the symbol the head is reading.

## Why It Matters

Turing machines are critical in the study of theoretical computer science. They allow us to answer fundamental questions about computation:

- **What can (and cannot) be computed?** It helps mathematically define computability and proves that some problems cannot be solved by any algorithm.
- **Computational Complexity:** It serves as the baseline model used to analyze the efficiency of algorithms.
- **Turing Completeness:** If a programming language or hardware system is powerful enough to simulate a basic Turing machine, it is considered "Turing-complete," meaning it can compute anything a modern universal computer can.

CIT

*A Turing machine* (TM) is a generalization of a PDA which uses a tape instead of a stack. Turing machines are an abstract version of a computer - they have been used to define formally what is *computable*. There are a number of alternative approaches to formalize the concept of computability (e.g. called the □-calculus, or □-recursive functions, ...) but they can all be shown to be equivalent. That this is the case for any reasonable notion of computation is called the *Church-Turing Thesis*.

## 10 Universal Turing-Machines,

In computer science, a **universal Turing machine** (**UTM**) is a Turing machine capable of computing any computable sequence,[1] as described by Alan Turing in his seminal paper "On

Computable Numbers, with an Application to the Entscheidungsproblem". Or, in other words, a Turing machine that is capable of simulating any other specialized Turing machines.

Common sense might say that a universal machine is impossible, but Turing proves that it is possible.

## 11 Church's thesis

The Church-Turing thesis (formerly Church's thesis) asserts that any function that can be calculated by an effective, algorithmic method is computable by a Turing machine. Proposed by Alonzo Church in 1936, it identifies intuitive "effective calculability" with rigorous mathematical definitions like lambda-definability, recursive functions, or Turing machine computation. Alonzo Church proposed that a function is effectively calculable if and only if it is lambda-definable (or recursive)

It serves as a foundational principle in theoretical computer science, helping to define the limits of what computers can solve. While Church's original thesis focused on lambda calculus, Turing's version focused on machine computation. Both are considered extensionally equivalent, often combined into the Church-Turing Thesis.

## 12 Solvability and Decidability.

### 12.1 Solvability

Solvability refers to the capability of a problem, equation, or system to be solved or resolved, ind icating whether a solution exists.

Definition and Context

- **General Definition**: Solvability is the condition of being solvable, meaning that there exists at le ast one solution that satisfies the conditions of a problem or equation. In mathematics, a problem is considered solvable if it can be resolved through established methods or techniques.
- **Mathematical Application**: In mathematics, solvability often pertains to equations or systems of equations. For example, a quadratic equation is solvable if it can be factored or if its roots can be

found using the quadratic formula. The concept is crucial in fields such as algebra, calculus, and differential equations.

**Related Concepts**

- **Solvable Problems**: These are problems that can be addressed and resolved using logical reasoni ng or mathematical methods. For instance, a solvable riddle or puzzle is one that has a clear answ er or solution.
- **Historical Context**: The term "solvability" has been in use since at least the late 1600s, indicatin g its long-standing relevance in discussions about problem-solving and mathematical theory.

**Importance**

Understanding solvability is essential in various fields, including mathematics, computer science, and economics, as it helps determine whether a given problem can be effectively addressed and what methods may be employed to find a solution.

In summary, solvability is a fundamental concept that plays a critical role in problem-solving across multiple disciplines, emphasizing the importance of identifying whether solutions exist for specific challenges.

## 12.2 Decidability

Decidability in computer science and logic refers to whether a problem can be solved by an algorithm (or Turing machine) that always terminates with a correct "yes" or "no" answer in a finite number of steps. A problem is decidable if such an algorithm exists; otherwise, it is undecidable.

Key aspects of decidability include:

- **Decidable Languages:** A language is decidable (or recursive) if a Turing machine can accept all strings in the language and reject all others, always halting.
- **Turing Machines:** Decidable problems are solved by algorithms that never loop infinitely.
- **Undecidability:** Problems for which no such algorithm can exist, such as the Halting Problem.

- **Semidecidability:** A weaker form where an algorithm might only guarantee a correct "yes" answer but may loop forever if the answer is "no".

Common examples include:

- **Decidable:** Membership in context-free languages, checking if a DFA accepts a string.
- **Undecidable:** Determining if a program will eventually halt (Halting Problem), determining if a Turing machine accepts a given string.

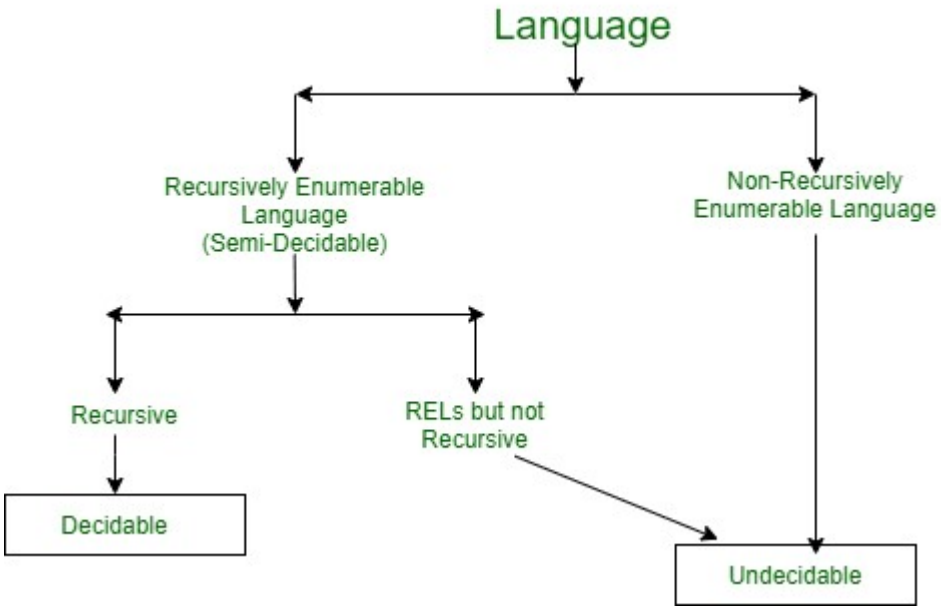



Fig. Decidable and Undecidable Languages

**Undecidable**

A formally stated problem is Undecidable if no total recursive function and thus, no Turing machine that always halts can be constructed to decide the problem, usually true or false.

Let us fix a simple alphabet Σ = **{0,1}**. As computer scientists we are well aware that everything can be coded up in bits and hence we accept that there is an encoding of TMs in binary. i.e. given a TM ***M*** we write ⌈M⌉∈ **{0, 1}*** for its binary encoding. We assume that the encoding contains its length such that we know when subsequent input on the tape starts.

Now we define the following language:

$L_{halt}$ = **{⌈*M*⌉*w* | ⌈*M* holds on input *w*}**

It is easy to define a TM which accepts this language: we just simulate ***M*** and accept if ***M*** stops. However, Turing showed that there is no TM which decides this language. To see this let us assume that there is a TM ***H*** which decides ***L***. Now using ***H*** we construct a new TM ***F*** which is a bit obnoxious: ***F*** on input ***x*** runs ***H*** on ***xx***. If ***H*** says yes then ***F*** goes into a loop otherwise (***H*** says no) ***F*** stops. The question is what happens if we run ***F*** on ⌈***F***⌉? Let us assume it terminates, then ***H*** applied to ⌈***F***⌉⌈***F***⌉ returns yes and hence we must conclude that ***F*** on ⌈***F***⌉ loops??? On the other hand if ***F*** with input ⌈***F***⌉ loops then ***H*** applied to ⌈***F***⌉⌈***F***⌉ will stop and reject and hence we have to conclude that ***F*** on ⌈***F***⌉ will stop????? This is a contradiction and hence we must conclude that our assumption that there is a TM ***H*** which decides $L_{halt}$ is false. We say $L_{halt}$ is undecidable.

## References


- Afolorunso, A. A. and Kehinde Obidairo (2011). Formal Languages And Automata Theory, National Open University Of Nigeria.
- Elaine Rich (2007). Automata, Computability and Complexity: Theory and Applications.
- D. Chandrasekhar Rao, Kishore Kumar Sahu and Pradipta Kumar Das . *THEORY OF COMPUTATION Lecture Nores,* Veer Surendra Sai University of Technology.